

MISAKA

A Post-Quantum BlockDAG Layer-1 with a Consensus-Native EVM and Objectively Verified Local and External LLM Work

The MISAKA Project

<https://github.com/MISAKA-BTC/misakas> · Explorer: <https://misakascan.com>

Whitepaper Version 2.0 – Research Specification – July 14, 2026

Abstract

Abstract. MISAKA is a post-quantum, PQ-only Layer-1 derived from *rusty-kaspa*. It retains GHOSTDAG BlockDAG consensus and a UTXO ledger while replacing classically vulnerable consensus-surface primitives with ML-DSA-87 authorization, a 64-byte keyed BLAKE2b-512 consensus identity, and an additive lattice accumulator. It further integrates a selected-parent EVM execution lane and a DNS-style stake overlay that gates deep reorganizations and supplies past-relative consensus randomness.

Version 2 adds PALW, a proposed audited-compute lane in which deterministic large-language-model inference and its GEMM schedule can become a bounded component of block work. PALW does *not* let a language model choose parents, transactions, timestamps, or validity rules. Expensive inference is executed before block production, committed by providers, published as leaves, selected for post-commit audit by future randomness, and matured into one-shot work tickets. A block producer merely consumes an active ticket in an otherwise ordinary deterministic block template. Full nodes verify the ticket, its certificate, slot, fork binding, lane difficulty, authorization, nullifier, and compute cap without rerunning the model on the block-acceptance path.

The design explicitly assumes that the nominal provider pair may be controlled by one adversary. Replica agreement is therefore a consistency signal rather than proof of two independent executions. Positive credit is based on minimum adversarial execution cost, future-randomized auditability, slashable exposure, reward escrow, proof-class subcaps, a permanent hash-work floor, and deterministic GHOSTDAG deduplication. Version 2 also makes ordinary local-model use a first-class PALW input: a local application can emit private or replicated compute receipts while it serves the user, while paid third-party requests carry a signed demand receipt and escrow. Token counts are never accepted as direct work. Self-private, self-replicated, and external-replicated jobs receive distinct objective multipliers, caps, fee rules, and privacy obligations. This paper defines the block-generation algorithm, the objective verification predicate, the DNS beacon and selected-parent state required to reproduce it, the attack model, and a staged activation process. Nonzero PALW work remains a testnet research target, not a claim of present mainnet activation.

What Version 2 changes

- Adds an LLM-work-backed block lane while preserving ordinary deterministic block construction.
- Defines objective verification as reproducible consensus verification from the selected-parent state and public commitments, not as blind trust in a committee certificate.
- Removes provider independence from the security assumptions and prevents $k = 2$ replica agreement from being counted as two executions.
- Introduces three separated future-randomness stages for pair assignment, audit selection, and ticket slots.
- Adds work, receipt, and ticket nullifiers; selected-parent-relative overlay state; lane-specific DAA; all-or-nothing compute caps; and reward escrow.
- Adds local-LLM mining modes and paid external demand without treating raw token count, requester identity, or self-declared “usefulness” as consensus work.
- Keeps PALW disabled or zero-credit until the complete validation, pruning, reorganization, audit, and adversarial-test gates are satisfied.

Contents

1	Introduction	4
1.1	Contributions	4
1.2	Reading guide	4

2	Status, Scope, and Claims	4
2.1	Normative vocabulary	4
2.2	Implementation status	5
2.3	Claim boundaries	5
3	Preliminaries and Notation	5
4	System Overview	6
4.1	Design goals	6
4.2	Non-goals	6
5	Post-Quantum Cryptographic Core	7
5.1	Transaction authorization	7
5.2	Hash64 identity	7
5.3	UTXO accumulator	7
5.4	Domain isolation	7
6	GHOSTDAG Consensus and the Permanent Hash Lane	7
6.1	Propagation and version safety	7
7	DNS Finality and Consensus Randomness	7
7.1	Two-resource deep-reorganization gate	7
7.2	Past-relative bond snapshots	7
7.3	Beacon V2	8
7.4	Overlay commitment root	8
8	Selected-Parent EVM and the Native-Asset Bridge	8
9	PALW Overview and Threat Model	8
9.1	Adversary capabilities	9
9.2	Security assumptions	9
9.3	No assumed provider independence	9
10	Deterministic LLM Work Objects	9
10.1	Runtime profile	9
10.2	Job precommit and work unit	9
10.3	Operation schedule and trace	10
10.4	Output commitment and privacy	10
10.5	Work nullifier hierarchy	10
11	Local LLM Participation and Paid Demand	10
11.1	Job classes and privacy modes	11
11.2	Interactive local workflow	12
11.3	Canonical compute units, not raw tokens	12
11.4	Demand receipt, escrow, and wash-trade resistance	12
11.5	Credit schedule, class subcaps, and quantumization	13
11.6	Fixed block reward and role allocation	13
12	Provider Assignment and Sybil-Resilient Exposure	14
12.1	Past-relative provider snapshot	14
12.2	Liability-weighted selection	14
12.3	Control domains	14
12.4	Pair reuse, probation, and exposure	14
13	Three Separated Randomness Stages	14
13.1	Receipt commit and reveal	14
13.2	All leaves before audit	15
14	Objective Audit, Dispute, and Certification	15
14.1	Assurance classes	15

14.2	Future audit draw	15
14.3	Canaries and real-job audits	15
14.4	Mismatch resolution	15
14.5	Certificate semantics	16
15	Ticket Creation and LLM-Work-Backed Block Generation	16
15.1	Batch state machine	16
15.2	Target slot	16
15.3	Checkpoint chain commitment	16
15.4	Expected lane difficulty and eligibility	16
15.5	Header V3 fields	17
15.6	Template algorithm	17
15.7	Authorization	17
16	Fast Consensus Verification	17
16.1	Canonical validation clauses	17
16.2	GHOSTDAG ticket deduplication	18
16.3	Pruning and IBD	19
17	Work Accounting, Compute Caps, and Lane DAA	19
17.1	Minimum adversarial cost	19
17.2	Component recurrence	19
17.3	Effective work and attack bound	19
17.4	Lane-specific DAA	19
18	Rewards, Slashing, and Liveness	19
18.1	Reward escrow and settlement	20
18.2	Expected fraud return	20
18.3	Objective evidence	20
18.4	Historic work	20
18.5	Degraded and halted modes	20
19	Security Analysis	21
19.1	Provider collusion	21
19.2	Fake or shortened computation	22
19.3	Ticket inventory and grinding	22
19.4	DNS and auditor compromise	22
19.5	Self-generated workload and token inflation	22
19.6	Circular external demand	22
19.7	Private-local downgrade attacks	22
19.8	Prompt injection and generated content	22
19.9	Denial of service	22
19.10	Privacy and auditability	22
20	Rollout, Test Gates, and Implementation Discipline	23
20.1	Activation stages	23
20.2	No-go conditions	23
20.3	Mandatory tests	23
21	Related Work	24
22	Conclusion	24
A	Consensus Object Sketches	24
B	Objective Verifier Pseudocode	25
C	Parameter Classification	26

1. Introduction

MISAKA is motivated by three pressures that are usually treated separately. The first is cryptographic. Classical elliptic-curve authorization and discrete-log-style accumulators do not retain their security against a sufficiently capable quantum adversary, while short consensus hashes lose security margin under Grover search. The second is structural. A linear chain converts propagation delay into orphaning pressure, whereas a BlockDAG can include concurrent honest blocks and order them through GHOSTDAG [1]. The third is computational. Modern networks consume substantial accelerator capacity for useful inference, but a naive attempt to call that activity “proof of work” fails because model execution is expensive to repeat, often private, hardware-sensitive, and easy for colluding providers to misreport.

Version 1 addressed the first two pressures and integrated an EVM lane. Version 2 adds a conservative answer to the third. It separates *work creation* from *block acceptance*. Providers execute a deterministic inference profile before a block exists. Their receipts, trace commitments, and public leaves are frozen before an unpredictable audit draw. Only after objective checks, a maturity delay, and economic exposure does a leaf become a ticket. A miner then uses the ticket at one deterministic DAA interval and one fork-bound checkpoint. Block acceptance remains a bounded sequence of hashes, signatures, membership proofs, state lookups, and integer comparisons.

This separation is essential. A language model is not a consensus oracle. Its generated text must never decide which transactions are valid, which parent is selected, or which fork wins. The model output is a service result whose canonical execution contributes a bounded resource claim. Consensus validates the claim’s protocol history and one-shot use.

1.1. Contributions

This paper makes the following design contributions.

1. It specifies a three-part Layer-1 consisting of a post-quantum UTXO ledger, a consensus-native selected-parent EVM, and a PALW audited-compute work lane over one GHOSTDAG.
2. It defines an objective PALW verifier whose inputs are the candidate block, its selected-parent-relative consensus view, and public or committed evidence. Local IP observations, operator reputation, and subjective model-quality judgements are excluded.
3. It treats provider A and provider B as potentially controlled by the same adversary. Replica matching is retained for fault detection and double liability but is not credited as two independent executions.
4. It binds jobs, provider assignment, audit selection, certificates, slots, lane difficulty, checkpoint commitments, authorization, and ticket consumption into a staged protocol using separated future randomness.
5. It specifies how ordinary local LLM use becomes mining input through self-private and self-replicated receipts, and how paid external requests obtain only a burn-backed demand bonus capped at one fifth of base compute.
6. It introduces conservative component-work accounting,

$$E = H + \min(C_{\text{strong}} + \min(C_{\text{replica}}, r_{\text{replica}}H), 4H), \quad (1)$$

where the permanent hash floor H remains indispensable and the replica class has its own stricter subcap.

7. It gives activation, rollback, pruning, reward, slashing, privacy, and adversarial-test requirements rather than pretending that a collection of structs is already a live protocol.

1.2. Reading guide

Sections 2–8 summarize the base MISAKA architecture. Sections 9–16 define LLM work, ticket creation, block construction, and objective verification. Sections 17–19 analyze accounting, economics, and attacks. Section 20 states the implementation and activation gates. Appendices provide object schemas, verifier pseudocode, and parameter classification.

2. Status, Scope, and Claims

2.1. Normative vocabulary

The terms **MUST**, **MUST NOT**, **SHOULD**, and **MAY** describe target consensus rules. A value marked *provisional* is a testnet or engineering proposal and is not a mainnet promise. A component marked *research* is not required for the current zero-credit stage.

2.2. Implementation status

Table 1. Version 2 status boundary.

Component	Version 2 statement	Status at publication
PQ UTXO consensus	ML-DSA-87 authorization, Hash64 identity, lattice accumulator	Base design / implementation track
GHOSTDAG and hash lane	Permanent consensus floor and selected-parent ordering	Base design
DNS finality overlay	Reorg gate and past-relative state commitment	Base design; V2 beacon hardening required
Selected-parent EVM	Consensus-native execution with delayed mergeset acceptance	Base design / staged implementation
PALW wire objects	Header-v3 fields, leaves, certificates, beacon, nullifiers	Reserved and partly implemented
PALW nonzero work	Positive GHOSTDAG compute credit	Not activated
Objective real-job audit	Future-randomized rerun/dispute path	Required before nonzero credit
Strong proof class	Succinct or transparent proof of inference	Future research

The publication describes a target hard-fork design. It must not be read as a claim that the current network accepts algo-4 blocks or that all validation clauses are wired into production. A finite activation fence is unsafe until the complete acceptance predicate, state rollback, template path, RPC serialization, pruning witness, and adversarial test suite agree.

2.3. Claim boundaries

MISAKA’s post-quantum claim covers the UTXO consensus authorization and consensus identity described in Section 5. It does not imply that every transport session, EVM account, external dependency, or optional trusted execution environment is post-quantum.

PALW’s objective-verification claim has a similarly precise scope. A full node can determine whether a block satisfies the PALW protocol without trusting private network telemetry or a model provider’s unsupported assertion. This does not mean that a replica certificate is a mathematical proof that two physical GPUs ran the complete model. For the replica class, safety is probabilistic and economic: future-randomized audit, public commitments, slashable exposure, conservative credit, and caps make fake work costly and bounded. A future strong-proof class may offer a different assurance model and must be accounted separately.

3. Preliminaries and Notation

Byte-string concatenation is written $\parallel$. Hash domains are length-delimited and network-bound. $\text{Hash64}_k(m)$ denotes keyed BLAKE2b-512 under domain key k . $\text{MerkleRoot}(S)$ denotes a canonical Merkle root of a sorted or index-addressed set S . All consensus integers have a fixed width and encoding; arithmetic that may overflow is invalid rather than saturating unless a rule explicitly says otherwise.

A BlockDAG is a directed acyclic graph $G = (V, E)$ whose vertices are blocks and whose edges point from a block to its parents. GHOSTDAG selects a well-connected blue cluster and assigns each block a selected parent $\text{sp}(B)$, a blue score, and cumulative blue work [1]. The selected chain induces an order over the past. A block’s mergeset contains its selected parent and eligible anticone blocks merged at that step.

For PALW, we distinguish:

Work unit. A canonical deterministic inference quantum under a frozen profile and operation schedule.

Candidate leaf. A public commitment to one work unit and its provider claims, not yet eligible for block work.

Active ticket. A candidate leaf that passed the required protocol, audit, certificate, and maturity transitions.

Source block. A block that consumes a ticket. The ticket may receive credit only once in GHOSTDAG consensus order.

Past-relative view. State resolved as of a candidate block’s selected parent, not from the node’s current virtual tip or mempool.

Objective predicate. A deterministic predicate reproducible by any validating node from consensus data and cryptographic evidence.

4. System Overview

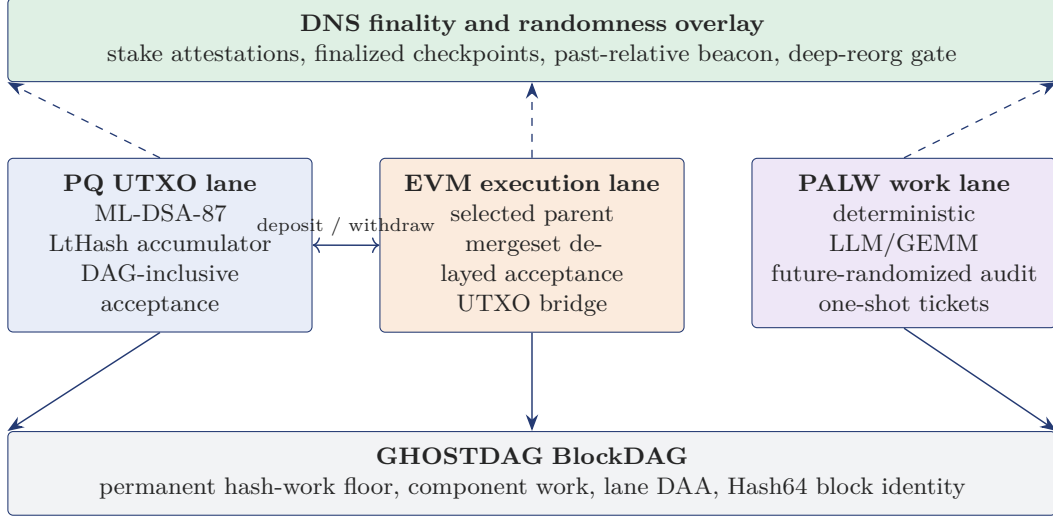


Figure 1. MISAKA Version 2 architecture. The UTXO and EVM lanes execute state transitions. PALW contributes a bounded, pre-certified work component. The DNS overlay provides finalized anchors, stake-gated reorganization protection, and consensus randomness.

The three lanes share block identity, selected-parent state, propagation rules, and GHOSTDAG ordering, but they do not share arbitrary trust assumptions. The UTXO lane is PQ-only. The EVM lane has a separate secp256k1 account domain unless a future PQ account format is introduced. PALW does not execute stateful smart contracts and does not grant providers permission to alter block content. Its output is a ticket reference and a bounded work increment.

4.1. Design goals

1. Preserve a permanent, independently verifiable hash-work floor.
2. Keep ordinary block validation fast and bounded; expensive inference must not be on the acceptance hot path.
3. Make every validity-relevant PALW input past-relative and reproducible after reorganization, restart, and pruning import.
4. Prevent hidden-leaf, ticket-stockpiling, fork-reuse, and first-seen-order attacks.
5. Treat Sybil identity as an economic exposure problem, not a solved identity theorem [2].
6. Separate model usefulness, service quality, and consensus work. A semantically poor but canonically executed job may be valid work; a semantically excellent but unverifiable private job receives no consensus credit.
7. Allow a local application to participate without exposing every prompt, while ensuring that strict non-disclosure receives zero work and that any positive private credit is backed by a committed future-audit path.
8. Make external-demand uplift economically non-circular: a requester signature alone is insufficient, and bonus credit is bounded by an irreversible fee burn and by selected-parent-relative epoch caps.

4.2. Non-goals

The design does not prove that two provider keys represent two people, companies, racks, or GPUs. It does not allow arbitrary private inference to receive positive credit without an audit-capable or strong-proof path. It does not treat TEE attestation as a consensus root. It does not award external-demand credit

merely because a second wallet signed a request. It does not retrospectively remove historic GHOSTDAG work when fraud is discovered after finality. It does not claim that an LLM output is factually correct merely because its execution was canonical.

5. Post-Quantum Cryptographic Core

5.1. Transaction authorization

The PQ UTXO lane uses ML-DSA-87 under FIPS 204 [3]. Consensus and wallet encodings must commit to the network, transaction version, input index, script context, and sighash mode. Legacy ECDSA, Schnorr, and P2SH authorization paths are not representable in PQ consensus mode. Optional transport confidentiality may use a hybrid ML-KEM construction under FIPS 203 [4], but transport security is distinct from consensus authorization.

5.2. Hash64 identity

Block hashes, transaction identifiers, Merkle roots, UTXO commitments, parent references, PALW object identifiers, and overlay roots use 64-byte, domain-separated BLAKE2b-512 digests [5]. The keyed mode is used for domain separation rather than secrecy. Length prefixes and version tags prevent two object families from sharing an ambiguous preimage.

5.3. UTXO accumulator

The UTXO set is committed through an additive lattice accumulator following the LtHash engineering line [6]. Additions and deletions are incremental, while the 64-byte header commitment binds the complete set. The accumulator’s algebra, serialization, and overflow behavior are consensus rules; an implementation may optimize representation but not change the resulting commitment.

5.4. Domain isolation

The EVM account signature domain, PALW provider signatures, DNS bond signatures, ticket authorization, and UTXO transaction signatures are distinct. A valid signature in one domain is not valid in another. This prevents a provider receipt, for example, from being replayed as a block authorization simply because both happen to use the same PQ signature algorithm.

6. GHOSTDAG Consensus and the Permanent Hash Lane

MISAKA inherits GHOSTDAG ordering and blue-work tip selection from Kaspas [1]. The base hash lane remains the security floor even when PALW is active. A block header carries a Layer-1 algorithm identifier, and a consensus-critical Layer-0 finalizer hashes the complete self-delimiting preimage into a 512-bit value.

For the live hash lane, the conceptual construction is

$$L_1 = \text{BLAKE2b512}(x) \parallel \text{SHA3-512}(x), \quad (2)$$

$$p_{512} = \text{BLAKE2b-512}_{k_{\text{pow}}}(\text{network} \parallel \text{algo} \parallel x \parallel \text{len}(L_1) \parallel L_1), \quad (3)$$

$$\text{accept} \iff \text{Uint512}(p_{512}) \leq \text{target}_{512}(\text{bits}). \quad (4)$$

The exact header preimage, target conversion, and work calculation are frozen by version. PALW does not weaken this check; it adds a second source class with its own validation and accounting.

6.1. Propagation and version safety

Unknown algorithm identifiers and malformed version-field combinations must fail closed without panics. Pre-v3 headers cannot smuggle nonzero PALW fields, and post-v3 fields enter the block-hash preimage in one frozen order. Template construction and validation share the same resolver rather than implementing parallel interpretations of activation.

7. DNS Finality and Consensus Randomness

7.1. Two-resource deep-reorganization gate

The DNS overlay is separate from block production. Active bonded validators attest to finalized anchors. A candidate history that exits a stake-confirmed anchor must satisfy the work rule and the overlay’s stake rule. This does not make ordinary short-range consensus BFT; it introduces a second resource at the deep-reorganization boundary.

7.2. Past-relative bond snapshots

For every PALW epoch E , the eligible DNS set is frozen at a cutoff before the corresponding commit window:

$$\mathcal{D}_E = \{(o_i, pk_i, s_i, a_i) \mid \text{bond } i \text{ is active at cutoff}(E)\}. \quad (5)$$

Here o_i is the bond outputpoint, pk_i the owner key, s_i effective stake, and a_i activation metadata. The quorum denominator is the total eligible stake $S_E = \sum_{i \in \mathcal{D}_E} s_i$, not merely the stake that chose to commit. A later unbond or slash does not rewrite the historic denominator, although objective evidence may affect current funds.

7.3. Beacon V2

Each participant commits to a secret opening:

$$c_i = \text{Hash64}_{k_c}(\text{network} \parallel \text{fork} \parallel E \parallel o_i \parallel r_i), \quad (6)$$

$$q_i = \text{Hash64}_{k_r}(\text{network} \parallel \text{fork} \parallel E \parallel o_i \parallel r_i). \quad (7)$$

After the reveal phase, the seed is derived from the actual openings, not from already-public commitments:

$$R_E = \text{Hash64}_{k_b}(R_{E-1} \parallel A_E \parallel \text{MerkleRoot}(\{q_i\}_{\text{valid}}) \parallel \text{MerkleRoot}(\{c_j\}_{\text{missing}}) \parallel E), \quad (8)$$

where A_E binds the finalized DNS checkpoint and eligible-set root. The missing-commitment root makes withholding accountable but does not eliminate last-revealer bias. The protocol therefore includes a reveal window, retransmittable reveals, degraded mode, and a zero PALW multiplier when the beacon cannot be considered healthy. Distributed randomness remains a protocol in its own right, not decorative hashing [7, 8].

7.4. Overlay commitment root

Every candidate block resolves the overlay as of its selected parent. Version 2 commits subroots rather than a mutable global snapshot:

$$O_B = \text{Hash64}_{k_o}(\text{v2} \parallel R_{\text{dnsBond}} \parallel R_{\text{beacon}} \parallel R_{\text{beaconFrontier}} \parallel R_{\text{provider}} \parallel R_{\text{batch}} \parallel R_{\text{audit}} \quad (9)$$

$$\parallel R_{\text{laneDAA}} \parallel R_{\text{nullifier}} \parallel R_{\text{checkpoint}} \parallel R_{\text{params}}). \quad (10)$$

The same state transition that updates UTXOs must atomically update these overlay roots. Reorganizations attach and detach block-indexed deltas; disk read failure is an error, not an invitation to substitute a zero root.

8. Selected-Parent EVM and the Native-Asset Bridge

The EVM lane binds each block’s EVM parent to its GHOSTDAG selected parent. A chain block executes the EVM payloads in its mergeset; its own payload is accepted by its selected child. This one-step delay makes each execution result an append-only function of the block’s parents. A virtual reorganization selects another already-computed state pointer instead of replaying an unbounded sequence merely because the virtual tip changed.

Each header commits separately to its own EVM payload and to the execution result of its accepted mergeset. The execution environment fixes chain identifier, fork rules, timestamp clamping, gas caps, base-fee arithmetic, skip classes, and state-root serialization. The design draws on deterministic optimistic execution literature while retaining a fixed consensus order [9].

The UTXO-to-EVM bridge preserves one native supply. Deposits lock UTXO value before it is minted into the EVM domain; withdrawals burn EVM value before creating a spendable UTXO. For all accepted states,

$$S_{\text{issued}} = S_{\text{UTXO spendable}} + S_{\text{bridge locked}} + S_{\text{EVM}} - S_{\text{burned}}. \quad (11)$$

EVM replay protection and fee conventions follow the relevant Ethereum standards [10, 11]. The EVM lane is not presented as PQ authorization unless and until a separate PQ account mechanism is activated.

9. PALW Overview and Threat Model

PALW is the audited-compute lane for deterministic LLM inference and its canonical GEMM/attention schedule. The acronym names a protocol family; the security claim is not that the model itself proves anything. The protocol converts pre-executed work into a one-shot ticket after commitments, random audits, and economic exposure.

Normative interpretation of “LLM block generation”

The LLM provider **does not construct consensus content**. It cannot choose the parent set, transaction order, timestamp, difficulty, selected checkpoint, or acceptance rule. A normal node

constructs the block template. The only PALW-specific input is a matured ticket reference whose slot and fork binding were already determined. The model output may be delivered to a requester, but its semantics do not enter block validity.

9.1. Adversary capabilities

The adversary may create many provider identities, control both members of a nominal pair, coordinate requesters, providers, auditors, and some DNS stake, reuse cached outputs, create large leaf inventories, withhold receipts or beacon reveals, exploit forks, censor audit data, and disguise common control through IP, ASN, funding, or reward-routing changes. It may also seek cheaper execution paths by skipping layers, shortening generation, changing quantization, exploiting nondeterministic kernels, or selectively computing only detectable canaries.

9.2. Security assumptions

The design assumes that hash and signature primitives remain secure; future seeds are not predictable before their commit deadlines; the deterministic runtime profile is reproducible; at least one required audit or dispute path is not fully controlled by the attacker; and the permanent hash lane continues to provide nonzero honest work. If any of these assumptions fails, the correct response is to reduce the PALW multiplier to zero, not to reinterpret a certificate optimistically.

9.3. No assumed provider independence

Invariant I-8v2: no assumed independence

Consensus MUST remain safe under the assumption that provider A and provider B are controlled by the same adversary. Different keys, bonds, IP addresses, or declared operator groups are not proof of independent execution.

Replica matching proves only that two slashable keys signed the same canonical receipt fields. It catches accidental faults, nondeterminism, and unilateral tampering. It does not establish that the work was run twice. Consequently, replica work is never normalized as “two GPUs worth” merely because two identities agreed.

10. Deterministic LLM Work Objects

10.1. Runtime profile

A consensus-recognized profile is identified by an exact manifest, not a marketing model name. At minimum it commits to:

- model-weight shards, tokenizer, architecture, quantization, and sampling policy;
- runtime image, compiler and kernel graph, arithmetic width, rounding and overflow rules;
- supported accelerator class, deterministic reduction order, batching, padding, and cache behavior;
- a fixed shape table, operation schedule version, output serialization, and reference test vectors.

TF32 shortcuts, stochastic rounding, nondeterministic reductions, driver-selected graph changes, and unpinned speculative decoding are not permitted within a deterministic profile unless their exact behavior is itself made canonical. Cross-vendor participation requires bit-exact conformance to the same vectors; otherwise it is a different profile and cannot be replica-matched.

10.2. Job precommit and work unit

A requester or scheduler first publishes a job precommit:

$$J = \text{Hash}_{64k_J}(\text{policy} \parallel \text{profile} \parallel \text{shape} \parallel \text{inputCommit} \parallel \text{auditCapsuleRoot} \parallel \text{feeTerms}). \quad (12)$$

The input may remain private, but the audit policy and capsule must already permit the verification path required for positive credit. The canonical job nullifier is

$$N_{\text{job}} = \text{Hash}_{64k_{N_J}}(J \parallel \text{profileId} \parallel \text{runtimeId} \parallel \text{shapeId}). \quad (13)$$

A work unit is a fixed quantum within the job, such as an indexed token range or a fixed operation bundle. Quantum boundaries cannot be chosen after the work outcome is known.

10.3. Operation schedule and trace

Each canonical operation receives an identifier

$$\text{opId}_j = \text{Hash64}_{k_{op}}(\text{layer} \parallel \text{token} \parallel \text{kernel} \parallel \text{expert} \parallel \text{shape} \parallel j). \quad (14)$$

The execution trace binds operation order and selected intermediate projections:

$$t_0 = \text{Hash64}_{k_t}(J \parallel \text{assignment} \parallel \text{challenge}), \quad (15)$$

$$t_j = \text{Hash64}_{k_t}(t_{j-1} \parallel \text{opId}_j \parallel \text{ioDigest}_j \parallel \text{accDigest}_j), \quad (16)$$

$$T = \text{MerkleRoot}(\{t_j\}_{j=1}^n). \quad (17)$$

The trace root is a commitment, not a succinct proof. Its value is that a later audit can demand openings at unpredictable operations and bisect a mismatch. A provider cannot invent a different schedule after seeing the audit seed without breaking the committed root.

10.4. Output commitment and privacy

The service output is salted:

$$O = \text{Hash64}_{k_o}(N_{\text{job}} \parallel \text{outputSalt} \parallel \text{canonicalOutputBytes}). \quad (18)$$

Public consensus data contains O , T , nullifiers, shape and profile identifiers, and a data-availability commitment. Raw prompts, hidden states, and full outputs are disclosed only under the job's audit policy. A private job that forbids every objective rerun or strong proof is valid as a service transaction but has PALW work multiplier zero.

10.5. Work nullifier hierarchy

Version 2 uses distinct nullifiers:

$$N_{\text{work}} = \text{Hash64}_{k_{Nw}}(N_{\text{job}} \parallel T \parallel O \parallel \text{quantumIndex}), \quad (19)$$

$$N_{\text{receipt}} = \text{Hash64}_{k_{Nr}}(N_{\text{work}} \parallel \text{providerBond} \parallel \text{assignmentId}), \quad (20)$$

$$N_{\text{ticket}} = \text{Hash64}_{k_{Nt}}(N_{\text{work}} \parallel \text{batchId} \parallel \text{leafIndex} \parallel \text{authorityKey}). \quad (21)$$

A semantic duplicate detector may flag suspicious rephrasings off consensus, but consensus deduplication relies on canonical commitments and nullifiers that all nodes can reproduce.

11. Local LLM Participation and Paid Demand

PALW is designed so that a user can run a supported local model for ordinary interactive use and, at the same time, produce mining-eligible receipts. The application path and the mining path share the same deterministic execution, but they do not share the same disclosure or credit policy. A user may keep a job completely local, permit a future confidential audit, replicate it to a randomly assigned provider, or process a paid request submitted by another user. The class is fixed before execution and is derived by validators from the committed job policy; it is not a label chosen in the block header.

Local-use invariant

A local prompt may produce PALW work only to the extent that the protocol can objectively verify its committed execution path. Strictly opaque local use remains a valid application activity but receives zero consensus work. Positive private credit requires a future-randomized audit capsule or a supported strong proof, and full Replica-class credit requires delivery to the beacon-assigned replica.

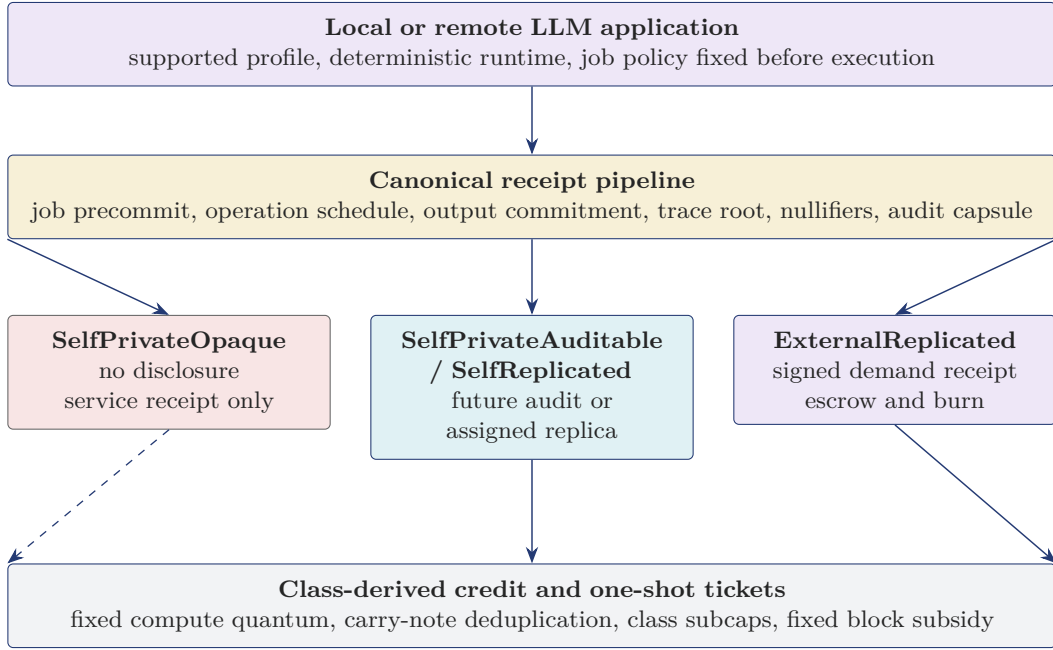


Figure 2. Ordinary local inference and paid external inference enter the same receipt pipeline but mature under different privacy, audit, fee, and credit rules.

11.1. Job classes and privacy modes

Job origin and assurance class are orthogonal. A job marked **SelfReplicated** still has zero work while it is merely **ReplicaMatched**; its mature multiplier applies only after the required audit and certificate state. Version 2 fixes the initial mature schedule in Table 2.

Table 2. Version 2 job classes. Multipliers apply only after the assurance requirements have matured.

Class	Data path	Required objective evidence	Mature multiplier
SelfPrivateOpaque	Prompt and output never leave the local device	Commitments only; no accepted rerun or proof path	0
SelfPrivateAuditable	Local execution; threshold-encrypted audit capsule or supported confidential rerun	Future-randomized opening, trace check, and certificate; TEE alone is insufficient	1/4
SelfReplicated	Local user is primary; encrypted job is routed to the beacon-assigned replica	Matching receipts, future audit, maturity, and both liabilities	1
ExternalReplicated	Paid requester escrow; beacon-selected primary and replica	SelfReplicated evidence plus valid demand receipt, settlement, and burn	$1 + B_{\text{ext}}/C_{\text{base}}$
ProtocolCanary	Protocol-generated indistinguishable audit job	Public canary transcript	0 work; audit payment only
StrongTransparent-Proof	Private or public job with a supported proof system	Proof verifies under its dedicated verifier and version	Separate class

A user choosing **SelfPrivateAuditable** accepts that a future audit draw may require the committed capsule to be opened to the selected audit path. Refusal or unavailability before maturity leaves the job at zero credit; refusal after accepting a positive-credit obligation is slashable according to the job policy. A user requiring unconditional local secrecy chooses **SelfPrivateOpaque**. This distinction prevents privacy from becoming an unverifiable work multiplier.

11.2. Interactive local workflow

The local client integrates PALW as a mining daemon around a supported model runtime:

1. Before execution, the client fixes the profile, maximum shape, job class, audit policy, fee terms, and a salted input commitment in a signed job precommit. The precommit must be accepted into the past-relative PALW registry before the applicable assignment and audit seeds. A result may still be served to the user if registration is late, but it is not mining-eligible.
2. The deterministic runtime performs the ordinary inference and emits the canonical operation schedule, output commitment, trace root, compute counters, and encrypted audit capsule. Raw prompt and answer bytes remain off chain.
3. For **SelfReplicated**, the scheduler derives the counterpart from the DNS beacon and active provider snapshot. The local user cannot choose provider B. The prompt is encrypted separately to B; B necessarily learns the plaintext inside its execution boundary unless a supported confidential-execution mode is used.
4. Receipts enter an all-leaf batch, are frozen before R_{audit} , and mature only after the objective audit, certificate, and evidence windows.
5. The local node accumulates mature compute quanta as owner-bound ticket leaves and consumes an eligible leaf only at its derived slot and chain commitment.

The precommit requirement prevents a client from running a large private inventory, observing later randomness, and publishing only favorable jobs. Implementations may reserve bounded interactive slots in advance, but each slot carries a fee, liability reservation, expiration, and unique sequence number; unused slots cannot be recycled after their assignment seed is known.

11.3. Canonical compute units, not raw tokens

Prompt and output token counts are useful telemetry but are not direct work. A request such as “emit the same token repeatedly” still incurs canonical decode work, but it does not become external demand merely by being long. For profile p , the consensus cost table defines minimum-adversarial coefficients for prefill tokens, decode tokens, attention blocks, active expert projections, and KV-cache updates:

$$C_{\text{base}} = \left\lfloor a_{\text{pre}}^{(p)} n_{\text{pre}} + a_{\text{dec}}^{(p)} n_{\text{dec}} + a_{\text{attn}}^{(p)} n_{\text{attn}} + a_{\text{expert}}^{(p)} n_{\text{expert}} + a_{\text{kv}}^{(p)} n_{\text{kv}} \right\rfloor_{Q_0}, \quad (22)$$

where $\lfloor x \rfloor_{Q_0} = Q_0 \lfloor x / Q_0 \rfloor$. Every count is derived from the frozen shape and canonical trace schedule, not copied from provider-supplied token fields. Coefficients are conservative lower bounds on the cheapest known compliant implementation and may only change by a versioned hard fork. The receipt may expose bucketed token metadata for service accounting, but validators recompute C_{base} from protocol objects.

Semantic value is deliberately absent from Equation (22). Self-generated work may be uninteresting and still be real canonical work, just as a hash nonce has no literary merit. The protocol instead prevents a claim of *external demand* from being free.

11.4. Demand receipt, escrow, and wash-trade resistance

An external job is mining-eligible only when a requester locks a unique escrow before provider assignment and signs a demand receipt bound to the job, profile, limits, expiry, and settlement policy.

Listing 1. External demand receipt sketch.

```
ExternalDemandReceiptV2 {
  version, network_id, job_commitment,
  requester_ephemeral_key, requester_nonce,
  requested_model_profile_id, max_input_tokens,
  max_output_tokens, escrow_outpoint,
  fee_amount, settlement_policy_id,
  expiry_daa_score, requester_signature
}
```

A second wallet is not evidence of independent demand. The requester may be controlled by the providers; consensus does not attempt to solve that identity problem. Instead, successful settlement applies the Version 2 fee schedule

$$(F_A, F_B, F_{\text{audit}}, F_{\text{burn}}) = \left(\frac{7}{20}F, \frac{7}{20}F, \frac{1}{10}F, \frac{1}{5}F \right). \quad (23)$$

This allocates 35% to the primary provider, 35% to the replica, 10% to the audit/data-availability pool, and 20% to an irreversible burn. Settlement occurs only after matching delivery and the required certificate state. On provider fault, unearned provider shares return to the requester and objective audit cost is recovered from provider liability; a post-assignment requester cancellation pays a bounded cancellation burn.

The paid-demand bonus is

$$\rho_E = \max\left(\rho_{\min}, 2 \text{EWMA}_8\left(\frac{R^{\text{PALW}}}{\max(C^{\text{mature}}, C_{\min})}\right)_{<E}\right), \quad (24)$$

$$B_{\text{ext}} = \min\left(\left\lfloor \frac{C_{\text{base}}}{5} \right\rfloor, \left\lfloor \frac{F_{\text{burn}}}{\rho_E} \right\rfloor, B_{\text{remaining}}^{\text{epoch}}, B_{\text{remaining}}^{\text{provider}}\right). \quad (25)$$

All inputs to ρ_E come from finalized past epochs and are resolved from the selected-parent view. During bootstrap or insufficient history, $B_{\text{ext}} = 0$. The first bound limits uplift to 20% of base compute. The second requires the irreversible burn to be worth at least the protocol's conservative estimate of the bonus. The factor two in ρ_E makes a circular self-request negative in expected value even if one adversary controls the requester and both providers. The remaining bounds limit concentration and sudden demand-credit bursts.

11.5. Credit schedule, class subcaps, and quantumization

After assurance maturity, job credit is derived as

$$C_{\text{job}} = \begin{cases} 0, & \text{SelfPrivateOpaque,} \\ \lfloor C_{\text{base}}/4 \rfloor, & \text{SelfPrivateAuditable,} \\ C_{\text{base}}, & \text{SelfReplicated,} \\ C_{\text{base}} + B_{\text{ext}}, & \text{ExternalReplicated,} \\ 0, & \text{ProtocolCanary.} \end{cases} \quad (26)$$

Version 2 additionally enforces

$$C_{\text{private}} \leq \frac{H}{32}, \quad C_{\text{demand}} \leq \frac{H}{32}, \quad C_{\text{replica}} \leq r_{\text{replica}} H, \quad (27)$$

inside the global compute cap. These are component caps in GHOSTDAG consensus order, not per-block suggestions. A future hard fork may lower them immediately; raising them requires a new activation review.

One ticket leaf represents a fixed quantum Q . Let $c_{\text{in}} < Q$ be a mature carry note:

$$n_{\text{leaf}} = \left\lfloor \frac{C_{\text{job}} + c_{\text{in}}}{Q} \right\rfloor, \quad (28)$$

$$c_{\text{out}} = (C_{\text{job}} + c_{\text{in}}) \bmod Q. \quad (29)$$

A carry note is non-transferable, owner-, profile-family-, job-class-, and proof-class-bound; it has a one-shot nullifier, contains separate base and demand components, remains strictly smaller than Q , and expires after two PALW epochs. It cannot be split, merged across classes, or reused on a fork. This preserves small-job usability without allowing one execution to be divided into arbitrarily many lottery leaves.

11.6. Fixed block reward and role allocation

Token count and external bonus do not change the coinbase amount. They affect the number of mature quanta and therefore the probability that a provider holds an eligible ticket. For a credited PALW source block with fixed subsidy R_B , Version 2 routes

$$R_{\text{primary}} = \frac{7}{10} R_B, \quad R_{\text{replica}} = \frac{1}{5} R_B, \quad R_{\text{audit/DA}} = \frac{1}{10} R_B. \quad (30)$$

The primary is the local provider for a self job or the beacon-selected primary for an external job; its key is the ticket authority. The replica beneficiary is fixed in the certified leaf. For a positive-credit class without a replica, the replica share is burned rather than reassigned to the primary. Audit/DA rewards are settled from objective participation records. Duplicate, revoked, over-cap, zero-credit, or red uses receive none of these provider rewards.

This allocation separates three economic facts: the requester service fee compensates execution and availability, the fixed block subsidy compensates scarce winning eligibility, and the demand bonus changes neither issuance nor the validity of the generated text.

12. Provider Assignment and Sybil-Resilient Exposure

12.1. Past-relative provider snapshot

The provider candidate set is frozen before the assignment seed. Each entry includes a bond outpoint, owner key, supported profile, runtime class, bond age, free slash liability, remaining capacity, probation state, pending exposure, and objective audit history. Local latency or network observations are absent.

12.2. Liability-weighted selection

Identity-count-weighted lottery is Sybil-positive: splitting one bond into many IDs increases the number of tickets in the hat. Version 2 instead defines selection weight by usable liability and capacity:

$$w_i = \min\left(\left\lfloor \frac{L_i^{\text{free}}}{\rho_{\text{unit}}} \right\rfloor, Q_i^{\text{capacity}}, Q_i^{\text{epoch}}, Q_i^{\text{pair}}\right). \quad (31)$$

A deterministic weighted sampling without replacement selects the pair from R_{assign} , the provider-set root, and the job precommit. Splitting the same total liability across identities must not increase aggregate selection probability beyond defined rounding bounds.

12.3. Control domains

Consensus-visible hard links, such as the same bond outpoint, owner key, device key, or prohibited role key, can reject a pair. Declared operator groups can impose conservative pair and exposure limits. IP, ASN, cloud region, funding graph, reward timing, and traffic fingerprints may help an off-consensus scheduler avoid correlated providers, but they cannot invalidate a block or trigger slashing because honest validators need not observe the same telemetry.

12.4. Pair reuse, probation, and exposure

A pair has a rolling exposure limit and a reuse cooldown. New providers begin in probation with low assignment and credit caps. For provider i , the required bond is linked to unresolved risk:

$$B_i \geq \gamma \frac{V_i^{\text{pending}}}{p_i^{\text{detect, lower}}}, \quad (32)$$

where $\gamma > 1$ is a safety margin, V_i^{pending} includes immature rewards and active-ticket liability, and $p_i^{\text{detect, lower}}$ is a protocol lower bound rather than a provider's optimistic estimate. Unbonding and reward release wait until all evidence windows close.

13. Three Separated Randomness Stages

Using one seed for provider assignment, audit selection, and ticket eligibility lets a registrant grind a job identifier for several favorable outcomes at once. Version 2 separates domains and deadlines.

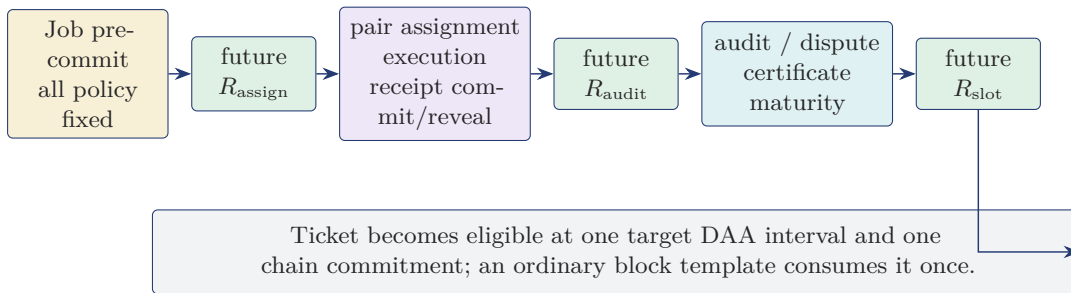


Figure 3. PALW timeline. Each future seed is revealed only after the values it selects over are committed.

The required ordering is

$$J \prec R_{\text{assign}}, \quad (\text{receipts, allLeaves, DA}) \prec R_{\text{audit}}, \quad (\text{certificate, activeSet}) \prec R_{\text{slot}}. \quad (33)$$

A value included after its selecting seed is known is invalid, even if every signature is otherwise correct.

13.1. Receipt commit and reveal

Providers first commit to their receipt hashes. Only after both commits are final do they reveal canonical receipts. This prevents one provider from waiting for the other's output and copying the exact fields without first fixing its own claim. It does not prove two executions, because one operator may control both commits; it does create two independently slashable signed statements.

13.2. All leaves before audit

A batch manifest fixes the leaf count, chunk count, profile, audit policy, bond reserve, and registration deadline. Every leaf is published or committed through indexed chunks before R_{audit} . Hidden leaves cannot be revealed only when favorable. Missing or duplicate chunks prevent the batch from advancing.

14. Objective Audit, Dispute, and Certification

14.1. Assurance classes

Table 3. PALW assurance classes. Multipliers and caps are hard-fork parameters.

Class	Required evidence	Interpretation	Initial credit
ServiceOnlyPrivate	output/trace commitments only	Service record; no objective rerun or strong proof	0
ReplicaMatched	two signed matching receipts	Consistency and double liability; audit pending	0
AuditedPrivate	committed private-audit capsule, future challenge, passed opening/dispute	Low-credit local class; strict opacity is not preserved if selected	1/4 origin multiplier
AuditedReplica	future-randomized audit policy, passed sample/dispute, matured certificate	Probabilistic/economic assurance under no-independence assumption	Conservative
TEE-assisted audited	AuditedReplica plus supported attestation	Operational signal only; TEE is not authority	No automatic uplift
StrongTransparent-Proof	Supported proof verifies the canonical computation	Separate future proof class	Separately capped

A miner cannot select a higher assurance class in the header. The class is derived from the certified leaf state and active protocol version. Job origin from Section 11 and assurance class are combined deterministically; either dimension can reduce the multiplier to zero.

14.2. Future audit draw

The audit plan is a pure function of R_{audit} , the committed leaf root, the provider-set root, and the policy:

$$A = \text{AuditPlan}(R_{\text{audit}}, R_{\text{leaf}}, R_{\text{provider}}, \text{policy}). \quad (34)$$

It selects leaves, trace positions, rerun sentinels, data-availability challenges, and auditors. A provider cannot identify the selected positions before committing the full trace and leaf inventory.

14.3. Canaries and real-job audits

Protocol canaries are delivered in the same envelope, padding class, timing window, and routing path as ordinary jobs. They detect providers that skip the runtime, but a canary-only system is insufficient because a provider may learn to classify them. Therefore, positive real-job credit requires that the job be capable of a post-commit independent check: requester-authorized rerun, threshold-escrow opening, a supported confidential rerun environment, or a strong transparent proof. If none is available, the leaf remains service-only.

14.4. Mismatch resolution

A third provider is not treated as a correctness majority. If A and B are one operator, $A = B \neq C$ says nothing about correctness. A mismatch moves the leaf or batch to **Quarantined**. Resolution uses a reference rerun, trace bisection, or strong proof. Interactive verification follows the general principle of narrowing a disputed computation to an objectively checkable step [12, 13].

For a disputed matrix multiplication $AB = C$, a Freivalds-style check can cheaply reject many false claims by sampling a field vector r and checking

$$A(Br) = Cr. \quad (35)$$

With canonical finite-field embedding, each independent round has bounded soundness error [14]. This is a dispute accelerator, not proof of the complete LLM, and it must not be advertised as one.

14.5. Certificate semantics

A DNS-auditor certificate signs objective facts, not a declaration that providers are independent or that generated text is true. A V2 certificate commits to:

- provider-set and eligible-auditor snapshot roots;
- job, pair-assignment, receipt-commit, receipt-reveal, leaf, and DA roots;
- audit policy, selected plan, responses, rerun/dispute result, and active-leaf bitmap;
- reserved liability and exposure roots;
- certificate version, network, fork, epoch, and maturity interval.

Votes are sorted by auditor identity, deduplicated, signature-checked, and weighted by the fixed eligible snapshot. Zero eligible stake cannot satisfy a zero-over-zero quorum. Conflicting votes are objective evidence.

15. Ticket Creation and LLM-Work-Backed Block Generation

15.1. Batch state machine

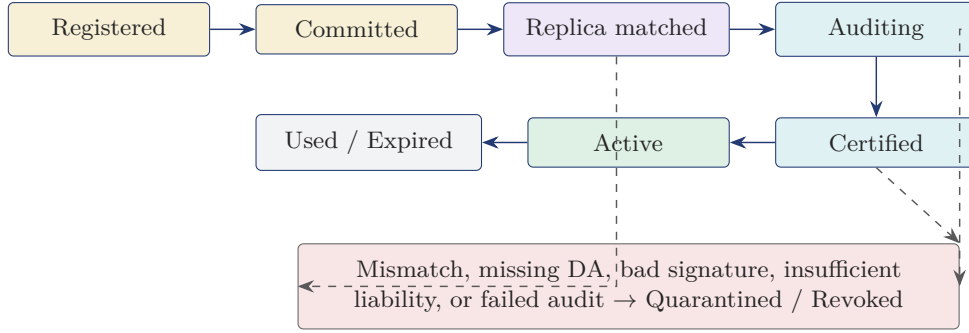


Figure 4. PALW leaf and batch lifecycle. Only an Active, mature, non-revoked leaf can become a block ticket.

15.2. Target slot

After the active set is frozen, R_{slot} assigns each leaf one target DAA interval:

$$S = \text{Slot}(R_{\text{slot}}, \text{batchId}, \text{leafIndex}, R_{\text{leaf}}, \text{slotWindow}). \quad (36)$$

The header copies S only so validators can compare it with the derived result. A miner cannot choose S . Checked arithmetic handles window endpoints and rejects overflow.

15.3. Checkpoint chain commitment

Let P_S be the latest DNS-finalized checkpoint at or before $S - L$, where L is a hard-forked lookback. The fork binding is

$$K_S = \text{Hash64}_{k_{\text{chain}}}(\text{network} \parallel \text{fork} \parallel P_S \parallel \text{dnsCertCommit} \parallel S). \quad (37)$$

No checkpoint means the PALW lane is halted for that slot. A zero placeholder is not a checkpoint.

15.4. Expected lane difficulty and eligibility

The selected-parent lane-DAA state determines $\text{bits}_S^{(C)}$. The ticket eligibility digest is

$$z = \text{Hash64}_{k_e}(R_{\text{slot}} \parallel K_S \parallel S \parallel N_{\text{ticket}} \parallel \text{proofClass} \parallel \text{paramsHash}), \quad (38)$$

and is eligible if $\text{Uint512}(z) \leq \text{target}_{512}(\text{bits}_S^{(C)})$. The canonical nonce equals a fixed 64-bit projection of N_{ticket} ; it is not a second grinding variable.

15.5. Header V3 fields

Table 4. Header-v3 PALW and component-work fields.

Field	Meaning and validation
blue_hash_work	Cumulative credited blue hash-lane work, recomputed from GHOSTDAG.
blue_compute_work	Cumulative credited compute work before the effective-work formula, recomputed and capped by consensus.
palw_batch_id	Active batch containing the referenced leaf.
palw_leaf_index	Canonical index within the committed leaf root.
palw_ticket_nullifier	First-class one-shot nullifier; binds nonce and authorization.
palw_epoch_certificate_hash	Hash of the active V2 certificate and maturity state.
palw_chain_commit	Derived lagged finalized-checkpoint binding from Equation (37).
palw_target_daa_interval	Derived slot from Equation (36).
palw_authorization_hash	Hash of the body-carried ML-DSA block authorization.
palw_proof_type	Derived assurance/proof-class discriminant.

15.6. Template algorithm

The block producer does not run the LLM while assembling a block. It selects an already active ticket and runs the same pure resolver used by validators.

Listing 2. Conceptual PALW template construction.

```

view = resolve_selected_parent_view(parents)
for ticket in view.active_ticket_inventory:
    if ticket.target_interval != next_daa_interval: continue
    binding = resolve_ticket_binding(view, ticket)
    if binding.expected_bits != template_bits: continue
    if binding.delta_compute > view.compute_headroom: continue
    if not one_shot_eligibility(binding): continue
    auth = sign_block_authorization(ticket.authority_key, header_preimage)
    return build_header_v3(view, binding, auth)
return build_hash_lane_header(view)

```

No local scheduling heuristic may change validity. It may choose among equally valid candidates, but every candidate must independently satisfy the objective predicate.

15.7. Authorization

The ticket authority signs a domain-separated authorization over the exact block preimage, network, fork, ticket nullifier, slot, chain commitment, certificate, and expiry. Reusing the ticket for a different header creates objective double-authorization evidence. The authorization object is body-carried and its hash is committed in the header.

16. Fast Consensus Verification

A validator resolves one immutable PALW view from the selected parent and then verifies the block. The procedure is fail-closed: missing state, unknown proof class, inconsistent component work, malformed signatures, arithmetic overflow, or unavailable required witness is a rule error.

16.1. Canonical validation clauses

An algo-4 block is valid only if all of the following hold atomically:

1. Header version and algorithm identifier are active for the actual selected-parent DAA state, not for an untrusted self-declared score alone.
2. Header shape is canonical; ticket-specific fields are nonzero where required and zero where forbidden.
3. The batch, manifest, leaf, receipt, certificate, maturity, audit, and revocation states exist in the selected-parent snapshot.

4. The leaf's profile, runtime, shape, proof class, provider bonds, and authority key are supported and match the certificate.
5. All signatures, vote uniqueness, stake weights, phase deadlines, and certificate roots verify.
6. The ticket and work nullifiers are canonical and not active in the selected-parent past.
7. The target slot S is derived from the committed active set and R_{slot} .
8. The lagged finalized checkpoint and K_S are derived, available, and equal to the header.
9. Lane DAA derives the exact expected bits; the miner cannot supply a different target.
10. The one-shot eligibility digest satisfies the derived target, and the nonce is canonical.
11. The body authorization hashes to the header field and verifies against the ticket authority.
12. The job class is derived from the precommit and certificate. A fully opaque job, unmatched receipt, or unaudited positive-credit claim is rejected or assigned zero credit according to the active stage.
13. For an external job, the demand receipt, unique escrow outpost, settlement state, fee burn, expiry, and requester signature verify; B_{ext} is recomputed from finalized past data.
14. Base compute, origin/assurance multipliers, carry-note transition, quantum count, class subcaps, and reward outputs are reproduced exactly.
15. The source block's compute increment is derived from proof class, job class, profile, shape, rollout stage, and minimum adversarial cost.
16. The full increment fits the deterministic compute headroom; partial credit is not permitted.
17. Recomputed `blue_hash_work`, `blue_compute_work`, `blue_work`, and `blue_score` equal the header and GHOSTDAG result.
18. The overlay post-state, active-nullifier update, reward class, and pruning witness commit to the same accepted result.

These clauses are activated together. Enabling only the easy header checks while deferring chain commitment, lane difficulty, cap, or authorization creates an invalid intermediate protocol.

16.2. GHOSTDAG ticket deduplication

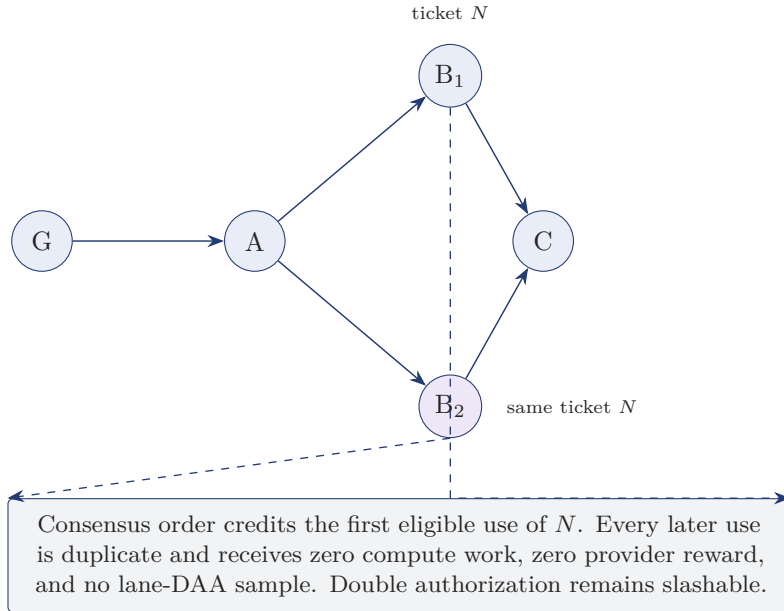


Figure 5. Ticket reuse on sibling branches is resolved by deterministic GHOSTDAG order, never by local first-seen time.

The active-nullifier view is block-indexed and reorganization-safe. Retention covers ticket expiry, private-fork replay, evidence windows, maximum supported reorganization, and pruning reconstruction. A simple two-minute cache is not a protocol if a ticket remains valuable longer than two minutes.

16.3. Pruning and IBD

A pruning bundle includes the finalized DNS certificate commitment, provider and eligible-set roots, active batch and leaf registry, audit result roots, lane-DAA state, active-nullifier root, component-work state, rollout parameters, and Merkle witnesses needed to validate the first post-pruning blocks. A pruned node must derive the same PALW predicate as an archival node. Header fields are evidence to compare, not trusted state merely because history was pruned.

17. Work Accounting, Compute Caps, and Lane DAA

17.1. Minimum adversarial cost

Replica credit is based on the cheapest plausible canonical execution available to an adversary controlling both provider identities, not on the honest provider’s claimed FLOPs, GPU count, electricity bill, or cloud invoice. A hard-forked profile/shape table maps a work unit to ΔC_{raw} . Measurements may motivate the table, but providers do not report their own weight.

For a source block b ,

$$\Delta C_b = \mu_{\text{rollout}} (\mu_{\text{assurance}} \mu_{\text{origin}} C_{\text{base},b} + B_{\text{demand},b}), \quad (39)$$

where $C_{\text{base},b}$ is derived by Equation (22), and all multipliers are rational consensus parameters with exact integer rounding. The demand term is nonzero only under Equation (25). The source block’s activation stage is frozen at acceptance so later parameter changes do not retroactively revalue old work.

17.2. Component recurrence

Let (H_p, C_p) be the component work of the selected-parent state after deterministic mergeset ordering. Hash source blocks add ΔH_b . Eligible PALW blocks add ΔC_b only if the entire increment fits:

$$H' = H_p + \sum_{b \in M_H} \Delta H_b, \quad (40)$$

$$C' = C_p + \sum_{b \in M_C}^{\text{consensus order}} \mathbf{1}[\Delta C_b \leq \text{headroom}_b] \Delta C_b, \quad (41)$$

$$\text{headroom}_b = \min \left(r_{\text{replica}} H_b - C_{\text{replica},b}, \frac{H_b}{32} - C_{\text{private},b}, \frac{H_b}{32} - C_{\text{demand},b}, 4H_b - C_{\text{total},b} \right). \quad (42)$$

Only caps relevant to the source class participate in the minimum. An over-cap block is not partially credited. Partial credit would make reward, DAA sampling, and security accounting depend on awkward fractional remnants. It is classified as valid-shadow or red according to the fork rule, with zero compute increment and zero provider base reward.

17.3. Effective work and attack bound

Equation (1) gives the effective blue work. For the replica class, Version 2 proposes the staged cap

$$r_{\text{replica}} \in \left\{ 0, \frac{1}{16}, \frac{1}{8}, \frac{1}{4} \right\} \quad (43)$$

for adversarial testnets. $1/2$ is a design ceiling for this revision, not a launch target. Even if every replica ticket were forged, the permanent hash component remains a large fraction of effective work; the exact takeover threshold depends on network work distribution and the DNS reorganization gate, so the paper does not collapse the result into one reassuring percentage.

17.4. Lane-specific DAA

Hash and PALW blocks have distinct sample windows and target rates. The PALW DAA samples only unique, active, blue, nonzero-credit source blocks. Duplicate, revoked, red, over-cap, and zero-rollout blocks do not adjust the PALW target. Sparse windows retain the previous bits or use a specified bootstrap target; timestamp clamps and maximum adjustment factors prevent burst release of stockpiled tickets from rapidly collapsing difficulty.

The slot rule and lane DAA are resolved from the same selected-parent view. No candidate block may influence the difficulty used to judge itself.

18. Rewards, Slashing, and Liveness

18.1. Reward escrow and settlement

Provider compensation is locked until at least ticket expiry, the fraud-evidence window, and the maximum supported reorganization horizon have passed. Version 2 uses the role allocation in Equation (30): 70% primary, 20% replica, and 10% audit/DA. The total subsidy is fixed by the monetary schedule and is independent of token count or demand class. If the certified class has no replica, the replica share is burned.

External service fees use the separate 35/35/10/20 settlement defined in Section 11. Provider shares and PALW subsidy shares remain escrowed against unresolved liability. Audit/DA claims are paid only from objective audit, retrieval, and dispute records. A job or block that later produces objective fraud evidence can slash the associated bond and all still-locked role rewards.

18.2. Expected fraud return

For a leaf with reward R , detection probability lower bound p , slashable bond and locked reward $B + L$, fees F , and attack computation cost K , a conservative model is

$$\mathbb{E}[\Pi_{\text{fraud}}] \leq (1 - p)R - p(B + L) - F - K. \quad (44)$$

Activation requires a parameter regime in which the right-hand side is negative under plausible low-cost attacks, not only under an honest two-GPU execution assumption.

18.3. Objective evidence

Slashable evidence includes conflicting signed receipt commits or reveals, double ticket authorization, invalid audit openings, provably inconsistent trace branches, duplicate auditor votes, false certificate roots, missing required DA responses, and role violations bound to on-chain keys. IP similarity, regional co-location, funding heuristics, and model-quality disagreement are not slashable evidence.

18.4. Historic work

Fraud discovered after finality does not retroactively delete historic GHOSTDAG work. Retroactive deletion would make validation depend on future discoveries and destabilize old checkpoints. Instead, unused tickets are revoked, bonds and locked rewards are slashed, providers and auditors are quarantined, future multipliers drop to zero, and evidence roots are committed. This is a forward containment model.

18.5. Degraded and halted modes

If the DNS beacon lacks quorum, finalized checkpoints are unavailable, audit DA is missing, or the selected-parent overlay cannot be reproduced, PALW enters a deterministic mode:

- **Healthy:** new batches may activate and the configured multiplier applies.
- **DegradedGrace:** no new activation; existing tickets have zero or explicitly reduced credit.
- **Halted:** algo-4 blocks are rejected; hash-lane block production continues.

The hash lane is therefore a liveness fallback as well as a security floor.

19. Security Analysis

Table 5. Principal attacks, protocol controls, and residual risk.

Attack	Primary controls	Residual risk
A/B Sybil collusion	No-independence assumption, one-execution cost floor, liability-weighted selection, pair/exposure caps, future audit	Physical common control cannot be fully identified
Fabricated output/-trace	Receipt commit/reveal, all-leaf publication, random openings, rerun/dispute, slash and escrow	Unsampled fraud remains probabilistic in replica class
Partial model execution	Frozen operation schedule, assignment challenge, trace openings, reference rerun	A cheaper unknown canonical-equivalent path can reduce the true cost floor
Canary-only honesty	Indistinguishable delivery, real-job audit capability, future selection	Perfect canary indistinguishability is not assumed
Leaf grinding	Precommit deadlines, per-leaf liability/fees, inventory quotas, future audit and slot seeds	High-capital actors can still buy more legitimate attempts
Same work in many leaves	Job/work nullifiers, canonical quantum indices, batch dedup, exposure accounting	Semantic duplicates beyond canonical commitment are hard to detect objectively
Ticket double use/fork reuse	Header nullifier, slot and chain commitment, authorization, GHOSTDAG-order dedup	Private forks can create slash evidence but not multiple credited uses
Malicious DNS certificate	Objective certificate fields, public votes, slashing, hash floor, replica subcap	Threshold DNS compromise can still damage the compute lane
Audit collusion	Future committee selection, public transcripts, reference dispute, role separation	If every allowed verifier is controlled, replica soundness fails and multiplier must be zero
Privacy leakage	Salted output, audit capsule, selective disclosure, service-only zero credit	Positive credit necessarily trades some auditability for privacy
Runtime differential	Exact manifest, class-specific matching, golden vectors, fail-closed profile activation	Hardware ecosystems may fragment into separate profiles
Self-query inflation	Actual canonical compute only, fixed quanta, per-provider exposure, class caps	Meaningless but real compute remains valid base work
External-demand wash trading	Unique escrow, 20% burn, bonus priced from finalized reward/CU history, 20% base uplift ceiling	Market-value estimation can lag abrupt changes
Carry-note replay or class laundering	Owner/class/profile binding, one-shot nullifier, two-epoch expiry, separate base/bonus components	Additional selected-parent state and pruning witnesses are required

19.1. Provider collusion

The central design change is to stop asking consensus to recognize independent humans from metadata. A/B agreement creates two liabilities and reduces accidental error, but work credit assumes one adversarially shared execution. If a future resource-bound proof can establish independent physical replication, it belongs to a stronger proof class with separate accounting. Filecoin’s proof-of-replication literature illustrates why counting replicas requires a resource-bound proof rather than merely multiple identities [15].

19.2. Fake or shortened computation

A provider may skip layers, truncate tokens, force expert routing, reuse KV state, reduce precision, or substitute a smaller model. Profile binding, operation schedule commitments, random trace openings, and reference reruns are designed to expose these deviations. However, a trace root alone is not proof, and an unknown shortcut that produces the same canonical output and openings at lower cost requires the cost table to be revised downward. Consensus credits the minimum known adversarial cost, rather than reported effort.

19.3. Ticket inventory and grinding

All leaves are fixed before audit randomness and all active tickets before slot randomness. Per-leaf bond reserve, fees, capacity quotas, and expiration make inventory costly. A participant may still obtain more legitimate draws by paying for more legitimate work; that is not the same as creating free hidden draws. The protocol's job is to prevent unpaid multiplicity and post-seed selection.

19.4. DNS and auditor compromise

DNS does not certify semantic correctness or provider independence. It certifies a deterministic transcript. Nevertheless, a threshold coalition could sign false facts if validators fail to verify them or if the protocol permits opaque facts. Version 2 therefore limits certificate fields to publicly reproducible roots and requires full nodes to verify the underlying state transitions. Hash work and class caps bound the maximum amplification, while slashing and public votes make compromise accountable. A total DNS compromise remains an explicit residual risk.

19.5. Self-generated workload and token inflation

PALW does not attempt to judge whether a local prompt was socially useful. An attacker may ask a model to emit repetitive text, but credit is derived from the canonical operation schedule and minimum adversarial cost, not from the requested token number. The attack therefore consumes real bounded compute and remains subject to provider exposure, ticket expiry, lane DAA, the Replica cap, and the private-work subcap. The protocol distinguishes this from falsely claiming that the same workload represents external demand.

19.6. Circular external demand

A miner can create a requester wallet and order work from itself. Identity heuristics cannot reliably prevent this. Version 2 instead makes the bonus conditional on a unique escrow and irreversible burn, with Equation (25) valuing bonus CU at no less than twice the finalized historical reward rate. Even if one controller recovers both provider shares, it loses the burn and cannot obtain positive expected value from the bonus alone. Base compute remains credited only because the canonical computation was actually performed and audited.

19.7. Private-local downgrade attacks

A client may try to register an opaque job and later relabel it as audited, or to refuse a selected opening after receiving provisional credit. Job class and audit policy are committed before execution, provisional receipts carry zero GHOSTDAG work, and only the matured certificate determines the multiplier. A missing opening cannot be normalized to an empty transcript. It either leaves the job service-only or creates objective slash evidence if the provider had accepted a positive-credit obligation.

19.8. Prompt injection and generated content

Prompt injection cannot alter consensus because the model cannot invoke node APIs, choose block fields, or sign a ticket authority message. Generated bytes are treated as committed output data. Any downstream application that interprets them does so outside the block-validity predicate. This separation should be preserved even if providers expose agentic tools for commercial service; agent permissions are not mining permissions.

19.9. Denial of service

PALW objects have strict size, count, recursion, signature, and allocation bounds. Parsers validate lengths before allocation. Unknown proof types are rejected. Expensive signature or Merkle checks occur only after cheap shape and activation checks. Invalid headers return structured rule errors rather than panicking. Overlay writes are atomic, and failures cannot be normalized to empty state.

19.10. Privacy and auditability

There is no general method to provide full objective verification of arbitrary private LLM inference without revealing data, trusting a confidential environment, involving the requester, or using a strong proof system.

Version 2 states this limit rather than hiding it behind the word “certificate.” A job owner chooses among service-only privacy, threshold/opening audit, confidential rerun, or a future strong proof. Consensus credit follows the assurance class.

20. Rollout, Test Gates, and Implementation Discipline

20.1. Activation stages

Table 6. Required PALW rollout stages.

Stage	Behavior	Replica cap
Disabled	Reject algo-4; no PALW state transitions	0
ShadowSidecar	Validate work objects off the DAG; measure runtime, DA, audit, and attacks	0
LocalReceiptOnly	Local applications emit Self/External receipts, demand escrows, and carry notes; no GHOSTDAG work	0
ConsensusVisibleZero-Credit	Algo-4 may be observed under strict quotas but adds no work, reward, or DAA sample	0
Adversarial-1	Full validation and rollback on a dedicated testnet	$H/16$
Adversarial-2	External red-team, stockpile, censorship, and all-compute-forged drills	$H/8$
Candidate	Only after sustained gates and independent review	at most $H/4$ initially
Version-2 ceiling	Absolute design ceiling for Replica class, not a target	$H/2$

A so-called weight-zero network is not truly zero influence merely because $\Delta C = 0$ if algo-4 blocks still affect blue score, anticone structure, DAA score, storage, or epoch timing. The safest first stage is a sidecar that does not enter the DAG. A later consensus-visible stage must specify every remaining effect explicitly.

20.2. No-go conditions

Nonzero replica credit is prohibited while any of the following remains unresolved:

- algo-4 can panic or receive GHOSTDAG credit before complete ticket validity;
- overlay, beacon, batch, nullifier, or lane-DAA state is global rather than block-indexed and reorganization-safe;
- DNS stake is a unit placeholder, reveal secrets do not affect the seed, or quorum uses committed stake as denominator;
- target slot, chain commitment, expected bits, authorization, eligibility, and full-increment cap are not active in one atomic predicate;
- template construction, P2P, gRPC, pruning, trusted data, and IBD do not round-trip all v3 fields and state witnesses;
- positive-credit jobs can refuse every real-job audit path;
- local job class, canonical compute units, demand escrow/burn, carry-note transitions, and coinbase role outputs are not independently recomputed by full nodes;
- the demand bonus can have positive expected value under self-request wash trading or lacks finalized-history/bootstrap safeguards;
- reward escrow and liability reservation are not enforced;
- adversarial tests have not demonstrated deterministic rollback to zero credit.

20.3. Mandatory tests

The acceptance gate includes header fuzzing over versions and algorithm IDs; activation boundary tests with forged self-declared DAA scores; processing-order permutation of commits, reveals, chunks, and votes; deep reorganization and crash recovery; sibling collective cap overflow; ancestor and fork nullifier reuse; sparse and burst lane DAA; reveal censorship; certificate equivocation; pruning import; cross-hardware golden vectors; parser allocation limits; end-to-end template-versus-validator equivalence; interactive

local-client precommit races; opaque-to-audited class downgrade attempts; demand-receipt replay; wash-trade simulations under common requester/provider control; fee-split and burn arithmetic; and carry-note double-spend across reorganizations.

Three core properties are:

$$\text{same selected-parent view} \Rightarrow \text{same seed, slot, chain commit, bits, cap, and verdict,} \quad (45)$$

$$\text{same valid message set in any arrival order} \Rightarrow \text{same committed overlay state,} \quad (46)$$

$$\text{accepted block} + \text{crash} + \text{restart} \Rightarrow \text{same UTXO and overlay roots.} \quad (47)$$

21. Related Work

MISAKA’s DAG consensus is inherited from GHOSTDAG [1]. The PQ primitives follow NIST’s ML-DSA, ML-KEM, and hash standards [3, 4, 16], with BLAKE2 domain-separated commitments [5]. The provider-identity analysis follows the fundamental observation that unconstrained distributed identity is vulnerable to Sybil multiplication [2].

PALW’s dispute model is related to optimistic verification and interactive computation [12, 13]; Freivalds checks provide a useful subroutine for matrix-product disputes [14]. Distributed randomness work informs the beacon design and its bias-resistance limits [7, 8]. Resource-bound replication systems such as Filecoin are a useful contrast: PALW Version 2 does not claim a proof of physical replication, so it credits the replica class conservatively and separately [15]. Succinct verifiable inference and zero-knowledge ML are potential future strong-proof classes, but they are not silently substituted for the current audited-replica design.

22. Conclusion

MISAKA Version 2 keeps a post-quantum UTXO consensus surface, GHOSTDAG ordering, a consensus-native EVM, and a DNS stake overlay, then adds LLM work without allowing a language model to become a consensus authority. Deterministic inference is performed before block production. Ordinary local-model use can emit self-private or self-replicated receipts, and paid external requests can add a narrowly bounded burn-backed demand bonus. Providers commit receipts and traces, all leaves are frozen before a future audit draw, objective audits and disputes mature a bounded work ticket, and a normal block template consumes that ticket once at a derived slot and fork commitment. Full nodes verify the protocol history, job class, fee burn, compute normalization, ticket ownership, and work caps cheaply and deterministically.

The design’s central security choice is deliberately unglamorous: it does not pretend to identify independent operators. It remains safe under common control by crediting no more than minimum adversarial execution cost, requiring audit-capable positive work, reserving liability, escrowing rewards, limiting concentration, and retaining both a permanent hash floor and a strict replica subcap. The remaining uncertainty is stated openly. Replica PALW is probabilistic and economic, not a mathematical proof of two physical executions. Until the complete state, audit, pruning, and adversarial gates are met, its correct consensus weight is zero.

A. Consensus Object Sketches

The following sketches are explanatory. Wire encodings require exact field widths, Borsh or equivalent canonical serialization, version domains, and bounded vectors.

Listing 3. Selected PALW V2 objects.

```
PalwJobPrecommitV2 {
  version, network_id, job_commitment, job_nullifier,
  job_class, model_profile_id, runtime_class_id,
  shape_id, quantum_count, audit_policy,
  audit_capsule_root, registration_deadline
}

PalwComputeReceiptV2 {
  job_commitment, job_class, model_profile_id,
  runtime_class_id, shape_id, canonical_compute_units,
  prompt_tokens_bucket, output_tokens_bucket,
  output_commitment, trace_root, operation_schedule_root,
  receipt_nullifier, provider_bond, signature
}
```

```

}

ExternalDemandReceiptV2 {
    job_commitment, requester_ephemeral_key, requester_nonce,
    requested_model_profile_id, max_input_tokens,
    max_output_tokens, escrow_outpoint, fee_amount,
    settlement_policy_id, expiry_daa_score, requester_signature
}

PalwCarryNoteV2 {
    owner_bond, profile_family, job_class, proof_class,
    base_units, demand_units, previous_note_nullifier,
    note_nullifier, created_epoch, expiry_epoch
}

PalwProviderSnapshotEntryV2 {
    bond_outpoint, owner_key_hash, runtime_class_id, status,
    bonded_amount, reserved_liability, free_liability,
    capacity_remaining, epoch_credit_remaining, pair_credit_remaining,
    bond_age, objective_audit_successes, objective_audit_failures
}

PalwReplicaReceiptV2 {
    assignment_id, provider_bond, job_nullifier, work_nullifier,
    output_commitment, trace_root, operation_schedule_root,
    receipt_da_root, execution_challenge_root, signature
}

PalwBatchCertificateV2 {
    batch_id, manifest_root, leaf_root, active_leaf_bitmap_root,
    provider_snapshot_root, pair_assignment_root,
    receipt_commit_root, receipt_reveal_root, audit_plan_root,
    audit_result_root, dispute_result_root, exposure_root,
    eligible_auditor_root, votes, maturity_interval
}

```

B. Objective Verifier Pseudocode

Listing 4. Conceptual block verifier.

```

verify_palw_block(block, selected_parent):
    view = resolve_palw_view(selected_parent)
    require view.rollout_stage.allows_algo4
    require canonical_header_shape(block.header)

    binding = lookup_active_leaf_and_certificate(view, block.header)
    verify_manifest_leaf_receipts_certificate(binding, view)
    verify_audit_and_maturity(binding, view)
    verify_provider_liability_and_proof_class(binding, view)

    job_class = derive_job_class(binding.precommit, binding.certificate)
    base_cu = derive_canonical_compute_units(binding.profile,
                                             binding.schedule)

    if job_class == ExternalReplicated:
        verify_demand_receipt_escrow_settlement_and_burn(binding, view)
        demand_cu = derive_demand_bonus(binding, view.finalized_history)
    else:
        demand_cu = 0
    delta_c = derive_class_credit(job_class, base_cu, demand_cu,
                                  binding.assurance, view.params)
    verify_carry_note_transition(binding, delta_c, view)
    verify_reward_output_vector(block.coinbase, binding, view.params)

```

```

require derive_slot(view, binding) == header.target_interval
require derive_chain_commit(view, binding) == header.chain_commit
require derive_lane_bits(view, binding) == header.bits
require canonical_nonce(binding.ticket_nullifier) == header.nonce
require one_shot_hash(binding, view) <= target(header.bits)
require verify_block_authorization(block, binding)
require not view.active_nullifiers.contains(binding.ticket_nullifier)

require delta_c <= derive_full_headroom(view, block.mergeset)

result = ghostdag_with_component_work(block, view, delta_c)
require result.component_fields == block.header.component_fields
require result.overlay_post_root == block.header.overlay_commitment_root
commit_atomically(result)

```

C. Parameter Classification

Table 7. Normative versus provisional PALW parameters.

Parameter	Classification	Version 2 rule
Hash floor	Normative	Permanent and nonzero
Global compute cap	Normative	Compute contribution no greater than $4H$
Replica class cap	Normative mechanism	Staged; initial nonzero candidate at most $H/4$, V2 ceiling $H/2$
Provider pair count $k = 2$	Normative for ReplicaExact matching	Consistency only, never two-execution credit
Audit probability	Hard-fork parameter	Must have a conservative lower bound used in bond economics
Local job multipliers	Normative V2 initial schedule	Opaque 0; private-auditable $1/4$; self-replicated 1
External demand uplift	Normative mechanism	At most $1/5$ of base, burn-priced, and subject to $H/32$ subcap
External fee settlement	Normative V2 initial schedule	35% primary, 35% replica, 10% audit/DA, 20% burn
PALW block subsidy routing	Normative V2 initial schedule	70% primary, 20% replica, 10% audit/DA; absent replica share burns
Carry note	Normative mechanism	Non-transferable, class-bound, one-shot, less than one quantum, two-epoch expiry
Reward split	Superseded for PALW V2	Version-1 77/8/15 proposal is not used by this revision
Model/profile labels	Non-consensus convenience	Exact manifest hash is consensus identity
Target BPS split	Provisional	Activated only with lane-DAA and adversarial measurements
TEE use	Optional	No authority and no automatic credit uplift
Strong proof system	Research	Separate class and cap when specified

References

- [1] Y. Sompolinsky, S. Wyborski, and A. Zohar. “PHANTOM GHOSTDAG: A Scalable Generalization of Nakamoto Consensus”. In: *Cryptology ePrint Archive* 2018/104 (2018). URL: <https://eprint.iacr.org/2018/104>.

- [2] J. R. Douceur. “The Sybil Attack”. In: *International Workshop on Peer-to-Peer Systems*. 2002, pp. 251–260. DOI: [10.1007/3-540-45748-8_24](https://doi.org/10.1007/3-540-45748-8_24).
- [3] National Institute of Standards and Technology. *FIPS 204: Module-Lattice-Based Digital Signature Standard*. Tech. rep. NIST, 2024. DOI: [10.6028/NIST.FIPS.204](https://doi.org/10.6028/NIST.FIPS.204).
- [4] National Institute of Standards and Technology. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard*. Tech. rep. NIST, 2024. DOI: [10.6028/NIST.FIPS.203](https://doi.org/10.6028/NIST.FIPS.203).
- [5] M.-J. O. Saarinen and J.-P. Aumasson. *The BLAKE2 Cryptographic Hash and Message Authentication Code*. Tech. rep. RFC 7693. IETF, 2015. DOI: [10.17487/RFC7693](https://doi.org/10.17487/RFC7693).
- [6] K. Lewi et al. “Securing Update Propagation with Homomorphic Hashing”. In: *Cryptology ePrint Archive* 2019/227 (2019). URL: <https://eprint.iacr.org/2019/227>.
- [7] E. Syta, P. Jovanovic, E. Kokoris-Kogias, et al. “Scalable Bias-Resistant Distributed Randomness”. In: *IEEE Symposium on Security and Privacy*. 2017, pp. 444–460. DOI: [10.1109/SP.2017.45](https://doi.org/10.1109/SP.2017.45).
- [8] League of Entropy. *drand: Distributed Randomness Beacon*. 2020. URL: <https://drand.love/docs/overview/>.
- [9] R. Gelashvili, A. Spiegelman, Z. Xiang, et al. “Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing”. In: *arXiv preprint arXiv:2203.06871* (2022). arXiv: [2203.06871](https://arxiv.org/abs/2203.06871).
- [10] V. Buterin. *EIP-155: Simple Replay Attack Protection*. 2016. URL: <https://eips.ethereum.org/EIPS/eip-155>.
- [11] V. Buterin et al. *EIP-1559: Fee Market Change for ETH 1.0 Chain*. 2019. URL: <https://eips.ethereum.org/EIPS/eip-1559>.
- [12] J. Teutsch and C. Reitwiessner. “A Scalable Verification Solution for Blockchains”. In: *arXiv preprint arXiv:1908.04756* (2019). arXiv: [1908.04756](https://arxiv.org/abs/1908.04756).
- [13] H. Kalodner, S. Goldfeder, X. Chen, et al. “Arbitrum: Scalable, Private Smart Contracts”. In: *USENIX Security Symposium*. 2018, pp. 1353–1370.
- [14] R. Freivalds. “Probabilistic Machines Can Use Less Running Time”. In: *IFIP Congress* (1979), pp. 839–842.
- [15] Protocol Labs. *Filecoin: A Decentralized Storage Network*. 2017. URL: <https://filecoin.io/filecoin.pdf>.
- [16] National Institute of Standards and Technology. *FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. Tech. rep. NIST, 2015. DOI: [10.6028/NIST.FIPS.202](https://doi.org/10.6028/NIST.FIPS.202).